

Visão Geral do Código

Esse código foi escrito em Python dentro da plataforma [Anvil](#), que permite criar aplicações web com Python puro.

O objetivo do programa é:

1. Ler o valor da conta digitado pelo usuário.
 2. Validar se o valor é válido.
 3. Calcular os preços para diferentes formas de pagamento.
 4. Mostrar os resultados em labels.
 5. Exibir mensagens de erro caso o valor seja inválido.
-

Estrutura Geral

O código possui dois métodos:

1. `button_calcular_click()` → Executado quando o botão **Calcular** é clicado.
 2. `exibir_erro()` → Mostra uma mensagem de erro em todos os campos de resultado.
-

1. Método `button_calcular_click`

```
def button_calcular_click(self, **event_args):
```

O que esse método faz?

É o evento associado ao botão "Calcular".

Sempre que o usuário clicar no botão, o Anvil executará esse método automaticamente.

Parâmetros

`self`

Representa a instância atual do formulário (a tela do aplicativo).

Por meio de `self`, o código acessa os componentes da interface:

- `self.text_box_valor`
- `self.label_dinheiro`
- `self.label_pix`
- etc.

****event_args**

Contém informações do evento do clique.

Na maioria dos casos, não é necessário utilizar esse parâmetro.

2. Leitura da Entrada

```
valor = self.text_box_valor.text
```

O que acontece aqui?

O programa lê o valor digitado pelo usuário no componente `TextBox`.

Exemplo

Se o usuário digitar:

150

Então:

```
valor = 150
```

Importante: `type="number"`

Como o `TextBox` foi configurado como `number`, o Anvil retorna:

- Um número (`int` ou `float`) se o usuário digitar algo.
- `None` se o campo estiver vazio.

Exemplos

Conteúdo da caixa	Valor recebido
100	100
45.90	45.9
vazio	None

3. Validação: Campo Vazio

```
if valor is None:  
    self.exibir_erro("⚠ Digite o valor da conta!")  
    return
```

O que faz?

Verifica se o usuário não digitou nada.

Se estiver vazio:

1. Chama `exibir_erro()`.
 2. Interrompe a execução com `return`.
-

Exemplo

Se o campo estiver vazio, todos os labels mostrarão:

⚠ Digite o valor da conta!

4. Validação: Valor Menor ou Igual a Zero

```
if valor <= 0:
    self.exibir_erro("❌ Valor deve ser maior que zero!")
    return
```

O que faz?

Impede valores como:

- 0
 - -10
 - -100
-

Exemplo

Se o usuário digitar:

0

Todos os labels mostrarão:

❌ Valor deve ser maior que zero!

5. Função Interna `fmt()`

```
def fmt(v):
    return f"R$ {v:.2f}".replace('.', ',')
```

Objetivo

Formatar números como moeda brasileira.

Como funciona?

`f"R$ {v:.2f}"`

- `:.2f` → duas casas decimais.
- Adiciona o prefixo R\$.

`.replace('.', ',')`

Troca o ponto decimal por vírgula.

Exemplos

Valor	Resultado
10	R\$ 10,00
25.5	R\$ 25,50
123.456	R\$ 123,46

6. Cálculo em Dinheiro (10% de Desconto)

```
self.label_dinheiro.text = fmt(valor * 0.90)
```

Fórmula

Novo valor = valor × 0,90

Exemplo

Se a conta for 100:

- 10% de desconto = 10
- Valor final = 90

Resultado:

R\$ 90,00

7. Cálculo no PIX (5% de Desconto)

```
self.label_pix.text = fmt(valor * 0.95)
```

Fórmula

Novo valor = valor $\times$ 0,95

Exemplo

Conta de 100:

- 5% de desconto = 5
- Valor final = 95

Resultado:

R\$ 95,00

8. Débito (Sem Alteração)

```
self.label_debito.text = fmt(valor)
```

O valor é mostrado sem desconto e sem juros.

Exemplo

Conta de 100:

R\$ 100,00

9. Crédito em 1x (2% de Juros)

```
self.label_credito1.text = fmt(valor * 1.02)
```

Fórmula

Novo valor = valor $\times$ 1,02

Exemplo

Conta de 100:

- Juros = 2
- Total = 102

Resultado:

R\$ 102,00

10. Crédito em 2x (4% de Juros)

```
total2 = valor * 1.04  
self.label_credito2.text = fmt(total2 / 2)
```

Etapas

1. Calcula o total com juros.
 2. Divide por 2 para obter o valor de cada parcela.
-

Exemplo

Conta de 100:

- Total = 104
- Parcela = 52

Resultado:

R\$ 52,00

11. Crédito em 3x (7% de Juros)

```
total3 = valor * 1.07  
self.label_credito3.text = fmt(total3 / 3)
```

Etapas

1. Calcula o total com juros.
 2. Divide por 3.
-

Exemplo

Conta de 100:

- Total = 107
- Parcela = 35,67

Resultado:

R\$ 35,67

12. Método `exibir_erro`

```
def exibir_erro(self, mensagem):
```

Recebe uma mensagem e a mostra em todos os labels de resultado.

Lista de Labels

```
labels = [  
    self.label_dinheiro, self.label_pix, self.label_debito,  
    self.label_credito1, self.label_credito2, self.label_credito3  
]
```

Essa lista reúne todos os componentes que receberão a mensagem.

Laço `for`

```
for label in labels:  
    label.text = mensagem
```

Percorre cada label e altera seu texto.

Exemplo

Se a mensagem for:

```
"⚠ Digite o valor da conta!"
```

Todos os labels mostrarão essa mensagem.

Fluxo Completo do Programa

Usuário digita valor

```
graph TD
    A[ ] --> B[Clica em "Calcular"]
    B --> C[Lê o valor]
    C --> D[Campo vazio?]
    D -- Sim --> E[mostra erro e termina]
    D -- Não --> F[Valor ≤ 0?]
    F -- Sim --> G[mostra erro e termina]
    F -- Não --> H[Calcula todas as formas de pagamento]
    H --> I[Exibe resultados formatados]
```

Exemplo Completo com Valor = R\$ 200,00

Forma de Pagamento	Cálculo	Resultado
Dinheiro	$200 \times 0,90$	R\$ 180,00
PIX	$200 \times 0,95$	R\$ 190,00
Débito	200	R\$ 200,00
Crédito 1x	$200 \times 1,02$	R\$ 204,00
Crédito 2x	$200 \times 1,04 \div 2$	R\$ 104,00
Crédito 3x	$200 \times 1,07 \div 3$	R\$ 71,33

Conceitos de Programação Utilizados

- Funções e métodos
- Eventos (`click`)
- Validação de entrada
- Condicionais (`if`)
- Retorno antecipado (`return`)
- Laços (`for`)
- Listas
- Formatação de strings
- Operações matemáticas